Samuel James Baker IV

samuel.j.baker908@gmail.com | (804) 761-6551 | samueljbaker.dev | github.com/samueljbaker | linkedin.com/in/samuel-j-baker-iv

EDUCATION

University of Virginia, School of Engineering and Applied Science • 3.5/4.0 **Expected December 2026**
Bachelor of Science in Computer Science - Artificial Intelligence Focal Path, Engineering Business Minor
Machine Learning, Reinforcement Learning, Digital Signal Processing, Computer Systems, Theory of Computation

Rappahannock Community College • 4.0/4.0 **Aug 2019 - May 2023**
Associate of Arts and Science, Summa Cum Laude

EXPERIENCE

Scaffold Education • Software Engineer, Platform and Infrastructure • Charlottesville, VA **May 2026 - Present**
- Architected and shipped a self-hosted chemical engineering LMS (66 pages, 55 modules) in **daily production use by 90+ students**.
- Built the telemetry pipeline and instructor progress dashboard (**14,700+ per-question attempt events**); diagnosed session-expiry and reveal-forfeit bugs from the event data, then shipped server-derived completion and a backfill restoring progress for 30 students.
- Designed the identity system: HMAC-SHA256 pseudonymization with a custodian-held pepper, httpOnly HS256 JWT cookie sessions, role-gated instructor access, and a re-sealable identity map so only the grading professor can de-pseudonymize progress.

PROJECTS & RESEARCH

LALTO - Inverse Rendering for Artistic Parameter Recovery • UVA Capstone Research **Jun 2026 - Present**
- Built an end-to-end inverse-rendering system recovering a **9-parameter artistic color-grading vector from a single rendered image**: OpenGL compute path tracer as forward model, a **33,031-render labeled corpus** (seeded, bit-exactly reproducible, hash-stamped provenance), and a multi-head ResNet-18 regressor with periodic sin/cos and heteroscedastic Laplace-NLL heads.
- Ported the renderer's post-process chain to a differentiable PyTorch model that bit-matches the C++/GLSL original (100% of pixels within 1 LSB), then added per-image gradient refinement against the exact forward model (Adam to L-BFGS, quantizer-likelihood interval loss), cutting parameter **MAE 51% (0.0496 to 0.0242)** and re-render **LPIPS from 0.0049 to 2e-5**.
- Replaced a ResNet-18 predictor (2 of 8 thresholds) with per-image inversion through a differentiable PyTorch port of the renderer (within 1 code value), plus L-BFGS continuation with restarts. This passed **8 of 8 thresholds** and **cut mean error 62%**.

Tommy - Multi-Modal Quant Research • Charlottesville, VA **Feb 2026 - Jun 2026**
- Built a point-in-time financial research pipeline in Python/PyTorch over CRSP, Compustat, IBES, OptionMetrics, EDGAR, and FRED, integrating structured and unstructured data across **66 tickers and 489 filings**.
- Implemented signal and forecasting models spanning HAR-LSTM-GARCH volatility prediction, FinBERT + FAISS RAG filing analysis, TFT-GNN multi-horizon return forecasting, and hierarchical VAE regime detection.
- Ran a pre-registered validation protocol (purged expanding-window CV, deflated Sharpe, PBO); ablations isolated entropy-conditioned position sizing at **+0.12 Sharpe (95% CI [-0.01, +0.25])** and found its confidence signal uncalibrated.

ECSEngine - C++20 Game Engine and GPU Path Tracer • Charlottesville, VA **Jun 2025 - Present**
- Built a C++20 entity-component-system engine (SDL3, OpenGL 4.6, ImGui) with in-engine profiling and debug tooling.
- Implemented a GPU path tracer in OpenGL compute shaders (GLSL) with A-Trous wavelet denoising, sustaining **40-200 FPS at 1-8 spp** on an RTX 4070 laptop GPU.
- Redesigned component storage from hash maps to dense arrays, cutting 100k-entity add/remove from **99.8 ms to 11.8 ms (8.5x)** and system update from **39.8 ms to 6.93 ms (5.7x)**.

Eggs by the Dozen • University of Virginia **Mar 2024**
- Collaborated on a 5-member team to build a Dockerized full-stack parasite-screening app with Express.js and Python/OpenCV, **automating McMaster-slide analysis** for sheep and goat samples.
- Designed a classical CV pipeline (blur, shifted modified-Otsu thresholding, contour and ellipse/shape-color matching) for mobile, low-connectivity use with too little labeled data for CNNs; **classified every test sample correctly**.
- Improved thresholding from a fixed value to modified Otsu with a translational shift, enabling correct health-condition classification on all test samples while reducing testing cost and turnaround

TECHNICAL SKILLS

Languages: Python, C++, SQL, TypeScript/JavaScript, Java, HTML/CSS, C#, Rust
ML/AI: PyTorch, TensorFlow, MLflow, FastAPI, OpenCV, Hugging Face, Transformers, FAISS, NLP, time-series forecasting, feature engineering, model training/evaluation, inference optimization
Data/Backend: Node.js, Express, REST APIs, MongoDB, PostgreSQL, SQLite, AWS S3, React, Next.js, Tailwind CSS
Systems: Linux, Git, CUDA, OpenGL 4.6, Docker, GPU compute, SDL3, ImGui